An empirical analysis of monolithic versus microservices architecture in low resource student environment

Abstract:

Modern software engineering heavily debates the merits of monolithic versus microservices architectures. While microservices offer scalability , fault isolation and technology heterogeneity, they typically demand significant infrastructure , orchestration and hardware overhead. This paper presents an empirical comparison of medium scale web application implemented twice – first as a traditional tightly coupled monolithic application and second as a distributed microservices architecture containerized via docker. Both system were benchmarked under identical hardware constraints on consumer-grade hardware simulating concurrent user traffic scaling form 10 to 500 requests per second. Results indicate distinct performance trade offs regarding memory footprint, CPU utilization , network overhead , and p95 latency under stress. The findings offer practical guidance and architecture framework for solo developers , academic researcher and student engineers and software developers operating under strict hardware and cost limitations
.

1.introduction

1.1 background and motivation

The foundation design of a software system dictates its long term maintainability , scalability , deployment velocity and operational complexity. Historically software systems were predominantly constructed as monolithic arch , wherein all business logic , data access layer and user interfaces are bundled into a single , cohesive executable or deployable unit.

In contrast, contemporary enterprise software has largely shifted toward microservices architecture. This paradigm decomposes an application into a collection of loosely coupled,independently deployable service that communicate over the network via lightweight protocols(e.g. HTTP REST,gRPC or message queues). While microservices provide clear organisational advantages for large , distributed systems complexities , including network latency , data consistency challenges and heavy infrastructure requirements.

1.2 the student and resource - constrained dilemma

While corporate literature and cloud providers heavily advocate for containerized microservices orchestrated via heavy platforms like Kubernetes , student developers , open-source hobbyists and independent creators frequently operate under server constraints. These constraints includes

- limited local hardware resources (e.g. laptop with restricted RAM and without dedicated server grade multi core processor)
- zero or minimal cloud infrastructure budget
- single developer or small team project scope where organisational scaling of microservices are entirely negated

Despite these realities , there is a notable scarcity of rigorous , empirical literature evaluating the exact performance penalties and resource footprint incurred when running microservices architecture in low resource , local development environments.

1.3 Research objective and contributions

This paper aims to bridge this gap by providing an exhaustive empirical analysis comparing monolithic and microservices architecture. The primary contributions of this work are:

1. The Implementation of a a dual model system: constructing an identical business domain(a full stack task management and telemetry logging system) twice once as a unified monolith and once as an orchestrated multi service cluster
2. Standard Benchmarking Framework: Executing repeatable , stress - tested load evaluations measuring the precise system metrics (CPU utilisation , RAM consumption , network I/O , and request latency distributions)
3. Actionable decision matrices: providing empirical guidance to help student developers and independent engineers select the optimal architecture based on resource availability and project requirements.

2. Literature Review and Theoretical Framework

2.1 Monolithic Architecture Characteristics and Trade–offs

A monolithic application is build as a single unified unit. Typically , it consists of a database , a user interface and a server side application containing various modulus(e.g. Authentication , business logic , data validation).

- Advantages:
    - Simplicity of deployment: The entire application compiles or packages into a single artifact , simplifying deployment to a single server instance or serverless function.
    - In memory performance: communication between modules occurs via fast in-memory function calls or method invocation , bypassing network serialisation delays.
    - Easier debugging and testing: developes can run the entire system locally with a single command and attach debuggers seamlessly across all execution paths

- Disadvantages:
    - Tightly Coupled codebases: As the application grows , dependencies tangle , making the codebase increasingly difficult to maintain and understand.
    - Scaling bottlenecks: scaling requires replication of the entire application , even if only a single compute heavy submodule requires additional resources.
    - Single point of failure: a runtime crash caused by a memory leak or unhandles exception in one module brings down the entire application.

2.2 Microservices architecture: characteristics and trade-offs

Microservices structure an application as a suite of small services , each running in its own process and communicating via lightweight mechanisms, often an HTTP resource API or event stream.

- Advantages:
    - Independent deployability:Teams can update , deploy and scale individual services without coordinating release for the entire system.
    - Fault isolation: if a single service fails , other services can continue operating with graceful degradation
    - Technology Heterogeneity: Different services can utilise different programming languages , frameworks or database engines best suited for their specific domain problem.

- Disadvantages:
    - Operational complexity: Managing multiple codebase, database migrations , network configuration and distributed tracing requires advanced tooling.
    - Network latency and serialisation: in memory calls are replaced by network requests , introducing serialisation(JSON/XML) , deserialization and network transport overhead.
    - Distributed data consistency : enforcing ACID transaction across multiple independent database requires independent databases requires complex patterns like the saga pattern or two phase commits.

2.3 The Architectural spectrum in academic context

Existing academic literature extensively covers enterprise-scale deployment , yet little focus is given to local, resource constrained simulations. Understanding the precise baseline overhead of containerization runtimes (such as docker desktop) running multiple microservices runtimes concurrently on consumer hardware remains critical for academics and low budget software engineering.

3. System design and technical architecture

To ensure a fair and scientifically valid comparison , both architecture models were designed to fulfil identical functional requirements , manage the same data schemas and expose matching API contracts.

3.1 Domain Specification

The chosen domain is a task management and user telemetry system , featuring the following core entities and operations:

- **User service/Module :** Handle user registration , authentication (JWT token generation) and profile management.
- **Task service/module:** Handles CRUD operations for tasks assigned to specific users , tracking status , priorities and timestamps.
- **Logging / audit module:** Records system events and requests telemetry for analytics.

3.2 Monolithic Implementation design

- **Architecture Style:** Domain - driven decomposition with decoupled services communicating over HTTP REST
- **Technology Stack:**
  - API Gateway: Node.js/Express reverse proxy routing traffic.
  - User Service: Independent Node.js application managing user state and database.
  - Task Service:Independent Node.js managing task state and database.
- Containerization and Orchestration: each service encapsulated within its own optimized Dockerfile(using multi-stage build) and orchestrated using unified docker-compose.yml configuration file , leveraging internal Docker bridge network for service discovery

4. Methodology and Experimental Setup

4.1 Hardware and software environment

All benchmarks were executed in a controlled , isolated environment to minimize external background noise:

- Hosting machine: Consumer laptop featuring a multi-core processor (8 thread) and 16 gb RAM , running docker desktop.

- Runtime environment : autocannon(an HTTP benchmarking tool written in Node.js , capable of generating high concurrency with low client-side resource overhead).

4.2 Experimental phase

1. Baseline resource capture: Measure idle CPU utilization and RAM consumption immediately after application startup before any incoming traffic.
2. Ramp-Up Stress Testing: subject both architecture to escalating load steps over a sustained window culminating in 100 concurrent connections.
3. Data collection: Aggregate metrics including Requests per seconds(RPS) , Averaging latency , P95 latency percentiles and peak resource consumption (CPU and RAM)

5 Results and Empirical Benchmarking Data

5.1 Baseline REsource consumption(IDLE state)

metric	Monolithic architecture	Microservices Architecture	Observation
Active runtime process	1 host process	3 container instance + gateway	Microservices requires multiple isolated runtime environments.
Idle memory footprint (RAM)	~72 MB	~ 315 MB	Container runtime and independent Node heaps cause a ~x4.3x memory increase
Idle CPU utilization	<0.1%	~0.8%	Negligible baseline utilization for both

5.2 Load testing performance Metrics (100 concurrent Users)

Performance Indicator	Monolithic Architecture	Microservice Architecture	Analysis
Throughout (request/sec)	3420 req/sec	1180 req/sec	Monolithic handles ~2.9x more request due to zero network overhead
Average latency	24.5 ms	84.2 ms	Internal network routing and serialisation add delay
P95 latency percentile	41.2 ms	142.6 Ms	Tail latency spikes under containerized network routing
Peak CPU utilization	~24.5%	~68.1%	JSON serialisation across HTTP Transport consumes extra CPU cycle
Peak RAM consumption	~115 MB	~480 MB	Linear scaling of memory under high concurrent load

6. Discussion and Critical Analysis

6.1 Resource Cost vs. Architectural Modularity

The empirical data conclusively demonstrates that deploying a microservices architecture in a resource-constrained environment introduces a severe performance and resource penalty:
Memory Inflation: Running multiple isolated container environments requires duplicating runtime environments (Node.js binaries, node_modules dependencies, and internal memory heaps), resulting in a roughly 4.3x increase in baseline RAM.
Network Overhead: Every internal service-to-service communication hop transforms an efficient in-memory pointer reference into a serialized JSON payload transmitted over an internal virtual network bridge. This accounts for the sharp rise in p95 latency.

6.2 When the Monolith is the Superior Choice

For student projects, solo developers, proof-of-concept phase applications, and low-resource edge servers, the monolithic pattern remains superior. It maximizes hardware efficiency,

minimizes deployment configuration overhead, and allows developers to focus purely on business logic rather than distributed systems infrastructure debugging.

7. Conclusion and Future Work

This paper conducted an empirical evaluation comparing monolithic and microservices architectures within a low-resource student environment. Through controlled benchmarking across concurrent user traffic, we established that microservices architectures impose a heavy operational toll: a 4.3x increase in idle memory consumption, significantly higher CPU utilization due to network serialization, and elevated request latencies. For developers operating under strict hardware or budgetary constraints, the monolithic architecture provides optimal performance, lower complexity, and superior resource efficiency. Future work can explore replacing standard HTTP REST communication with binary serialization protocols such as gRPC to measure latency reductions.